Leveraging Large Language Model for Automating Code Review in Scientific Workflows: Replicating and Evaluating Traditional Approaches

1st Al Muttakin
Department of Computer Science
University of Saskatchewan
Saskatoon, SK, Canada
al.muttakin@usask.ca

2nd Tasnova Ahmed
Department of Computer Science
University of Saskatchewan
Saskatoon, SK, Canada
kjs901@usask.ca

3rd Banani Roy
Department of Computer Science
University of Saskatchewan
Saskatoon, SK, Canada
banani.roy@usask.ca

***Abstract*—Scientific workflows in fields such as bioinformatics and computational biology are often written in domain-specific languages (DSLs) like Nextflow. These DSLs make it easier to build complex pipelines but also raise challenges for assuring code quality. Rigorous code review can help, yet manual review is time-consuming and often inaccessible to contributors with limited programming experience. As s result, many software engineering studies have tried to automate the code review process by leveraging large language models (LLMs). In this study, we replicate a state-of-the-art code review model trained on general-purpose languages (e.g., Java, Python) and evaluate it on Nextflow pipelines to test whether it can generate review comments for DSL-based scientific workflows. The baseline model performs poorly on scientific workflow code across standard evaluation metrics. However, fine-tuning the model on workflow data yields large gains: BLEU and ROUGE improve by 442% and 300%, respectively; human ratings of relevance increase by 120%; and BERTScore and LLM-based evaluations show moderate improvements. A complementary user study indicates that automated review lowers perceived mental demand and effort across common review tasks. Together, these results suggest that domain-adapted LLMs can improve the quality of code review for scientific workflows and reduce barriers for new and non-expert contributors, supporting more inclusive and scalable scientific software development.**

***Index Terms*—Scientific Workflows, Code Review, Large Language Models, Domain-Specific Languages**

I. Introduction

Scientific workflows are central to data-intensive research in fields such as bioinformatics and computational biology, where teams assemble complex pipelines to transform raw data into reproducible findings. Many of these pipelines are authored in domain-specific languages (DSLs) inside workflow systems like Nextflow, Snakemake, and Galaxy because these tools help modularize steps, manage dependencies, and scale across clusters and clouds [1]–[3]. At the same time, DSL-based workflows introduce challenges that differ from those in general-purpose software. Their syntax and execution semantics are specialized, orchestration choices can silently affect results, and small mistakes can undermine reproducibility. Software engineering practices can address these risks. In the scientific community, authors have argued for bringing systematic code review into research to improve reliability and reproducibility, and have offered practical guidance and checklists for conducting reviews in research settings [4]–[8]. Related work in workflow engineering also shows that adopting testing and verification practices, such as test-driven development for Nextflow pipelines, can reduce regressions and harden workflows against subtle failures [9]. Building on this momentum, automated code review with large language models (LLMs) promises to provide timely, consistent feedback at scale, especially when expert reviewers are scarce or contributors have limited programming experience.

Automated code review is an active research area in software engineering. Modern code review is widely practiced in industry and open source, where it supports defect detection, knowledge sharing, and team awareness [10]. Recent studies explore LLM-based automation for complementary tasks—comment generation, review necessity detection, and code refinement—while also noting open challenges in evaluation and domain transfer [11]–[14]. However, this work has focused on projects written in general-purpose languages. To our knowledge, there is no prior study of automated code review for DSL-based scientific workflows, where the constructs, orchestration patterns, and domain assumptions differ sharply from mainstream programming. This gap matters because workflow repositories increasingly host collaborative development, yet review capacity is limited and many contributors come from non-software backgrounds [4]–[6].

In this paper, we study automated code review for scientific workflows authored in Nextflow. We replicate a recent LLM-based code review approach and evaluate it on a curated test set from ten widely used Nextflow pipelines. We then fine-tune the model on domain-specific data drawn from the same ecosystem and compare performance to the replicated baseline. Our evaluation combines automatic metrics widely used for review-comment quality—BLEU, ROUGE, and BERTScore—with an LLM-as-judge setup and a practitioner-focused assessment [15]–[17]. We also conduct expert interviews and a cognitive-effort study in which reviewers analyze workflow snippets with and without automated comments. At a high level, the

replicated baseline struggles with DSL-specific constructs and workflow semantics, while domain adaptation yields clear gains across automatic metrics and reduces perceived mental demand during review. These results motivate our research questions.

- **RQ_1 (*Replication*): How well existing code review models perform on DSL-based scientific workflows?** This question establishes a baseline for automated review on domain-specific workflow code. We replicate a recent LLM-based code review model trained on general-purpose languages (CodeReviewer) and apply it to Nextflow pipelines to see how far such models transfer without domain adaptation.
- **RQ_2 (*Finetuning*): What types of modifications are needed to execute replication packages?** This question tests whether targeted adaptation bridges the gap between general-purpose training and DSL-based workflows. We fine-tune the replicated model on our curated Nextflow review dataset and re-evaluate on the same held-out test set. The comparison to RQ1 uses the identical metrics and protocol to quantify gains attributable to domain-specific learning.
- **RQ_3 (*Expert's Opinion*): What challenges prevent the successful execution of replication packages?** This question evaluates practical impact for the scientific workflow community. We run a cognitive-effort study in which reviewers analyze Nextflow snippets of varying complexity with and without LLM-generated comments, and we measure perceived mental demand and effort. By comparing conditions, we estimate whether automated feedback reduces reviewer burden and offers helpful guidance to contributors with limited programming background, thereby lowering barriers to participation

II. Background & Related Work

In this section we describe the necessary background, related works and evaluation matrices required to follow this study.

A. Background

Scientific workflow management systems have become central to data-intensive research because they let scientists compose complex, multi-step analyses that run reliably on clusters and clouds. Systems such as Nextflow, Snakemake, and Galaxy make this possible by offering domain-specific abstractions for expressing tasks, data dependencies, and execution environments, often through a DSL that hides low-level orchestration details [1]–[3]. These benefits come with challenges. DSLs encode specialized syntax and execution semantics that differ from general-purpose languages, so small mistakes in process definitions, channels, or resource declarations can silently alter results. Drawing on software engineering can help. Evidence from the scientific community shows that adopting established practices—such as systematic review, explicit checklists, and test-driven development—improves reliability, readability, and reproducibility of research code [4]–[6], [9]. This cross-pollination motivates treating workflow scripts as first-class software artifacts that benefit from disciplined quality assurance.

Code review is a natural bridge between scientific workflows and software engineering. In software projects, review supports defect finding, knowledge sharing, and team awareness, and is now standard practice in both industry and open source [10]. Scientific code can realize similar benefits. Studies and guidelines aimed at research teams argue that routine review reduces errors, improves clarity, and fosters a culture of reproducibility, while acknowledging barriers such as time cost and uneven programming experience [4]–[8]. For DSL-based workflows, review also provides a venue to check orchestration concerns that typical unit tests may miss, such as dataflow between channels or container compatibility. In short, review complements testing in assuring the quality of Nextflow and related DSL pipelines. Fig 1 illustrates a real-world code review scenario in a public pull request from a Nextflow repository.

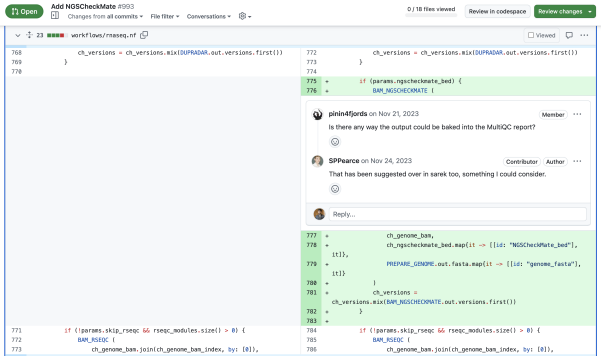

Fig. 1: Real life code review example.

Automated code review extends these benefits by offering timely, consistent feedback at scale. Early efforts relied on static analyzers and rule-based checks, which are precise but narrow. Recent work in software engineering has moved toward learning-based approaches. CodeReviewer pretrains on large corpora of review data and tackles tasks such as review-comment generation and quality estimation, showing gains over generic code models [11]. Other directions retrieve and adapt past comments [12] or fine-tune LLMs with parameter-efficient methods to produce higher-quality feedback [13]. Survey work maps this area and highlights persistent limitations, including language coverage, granularity, and evaluation practices [14]. None of these studies address the unique demands of DSL-based scientific workflows, where the semantics of channels, processes, and execution back ends differ markedly from general-purpose code—creating a natural testbed for domain adaptation.

Evaluation criteria play a key role in assessing automated review. N-gram overlap metrics like BLEU and ROUGE are common because they are simple and reproducible, but they reward surface similarity and can misjudge semantically correct rephrasings [15], [16]. Embedding-based measures such as BERTScore better capture meaning by comparing contextual token representations [17]. In our study, we use BLEU-4, ROUGE-L, and BERTScore to quantify lexical and semantic

alignment between generated and reference comments. We complement these with an LLM-based evaluator, inspired by prior LLM-as-judge designs, implemented using LLaMA-3-8B-Instruct to assess consistency, helpfulness, and fluency. Finally, we include human judgment: two authors rate roughly one-fifth of the test set on informativeness and relevance using a 1–5 Likert scale. This combination balances reproducibility with a closer approximation of practitioner expectations, while acknowledging that human assessment remains costly and that LLM-based judges require careful rubric design [14].

B. Related Works

Scientific workflows increasingly integrate large language models to reduce manual burden and to broaden participation by non-programmers. At a conceptual level, LLM-enabled systems treat scientific discovery as a computational process, combining language understanding with tool use, symbolic reasoning, and data access to plan and execute analyses [18]. Domain studies show how prompt design and modular architectures translate plain-language descriptions into executable scripts and configurations for simulation and data processing [19]. Benchmarking across workflow platforms suggests that strong proprietary LLMs outperform open-source baselines when given domain context, while also revealing risks such as hallucinations and system-specific misunderstandings that require human oversight [20]. Human-centered designs emphasize inclusion of domain experts in data science workflows, linking free-text intent to executable steps and improving provenance and traceability [21]. Emerging tools go further by generating full workflows and linking resources through knowledge graphs, lowering entry barriers while aiming to preserve rigor and reproducibility [22], [23]. These efforts motivate bringing automation to quality assurance tasks—like review—in the workflow setting.

Automated code review with LLMs has advanced quickly in traditional software engineering. CodeReviewer shows that review-specific pretraining yields better comment generation and quality estimation than general code models [11]. Retrieval-based recommenders provide a lightweight alternative by matching diffs to prior comments [12]. Parameter-efficient fine-tuning strategies adapt LLMs to the review domain with modest resources [13]. Beyond modeling, recent work analyzes weaknesses in current evaluation pipelines and datasets, noting overreliance on surface-form metrics, contamination risks, and noise in mined benchmarks [14]. Other studies enrich training data with reviewer-experience signals to improve actionability and specificity of generated feedback [24]. Together, these results point to the importance of domain-specific training and careful evaluation—two themes that carry directly into DSL-based workflows.

Scientific communities have begun adapting code review to research contexts. Empirical and guidance papers argue for integrating review into everyday research practice to improve error detection, clarity, and reproducibility, while recognizing barriers such as time, incentives, and uneven programming expertise [4]–[8]. Checklists translate software engineering heuristics into research-appropriate criteria covering reporting, running, reproducibility, reliability, robustness, readability, and release [6]. Complementary tooling, such as test-driven frameworks for Nextflow, demonstrates how software engineering methods can harden pipelines and reduce regressions [9]. Our work builds on this foundation by examining automated review for Nextflow specifically, assessing how far a replicated software-engineering model transfers to DSL workflows and whether domain adaptation improves comment quality and practical usefulness.

III. Motivating Example

A recent discussion in the Nextflow community shows why automated code review can make a real difference in scientific workflows. In April 2024, a user asked for a feature to surface FastQC read quality scores in the MultiQC "General Stats" table. A maintainer replied that a pull request already existed to address the request and invited community feedback on the in-progress branch. The user appreciated the update but declined to review the pull request, explaining that they were not confident with Python and would stay on the production branch while waiting for a release [25]. The pull request itself had been open for years, with recent activity but no merge, so the feature remained unavailable to end users [26]. Fig 1 shows this exchange.

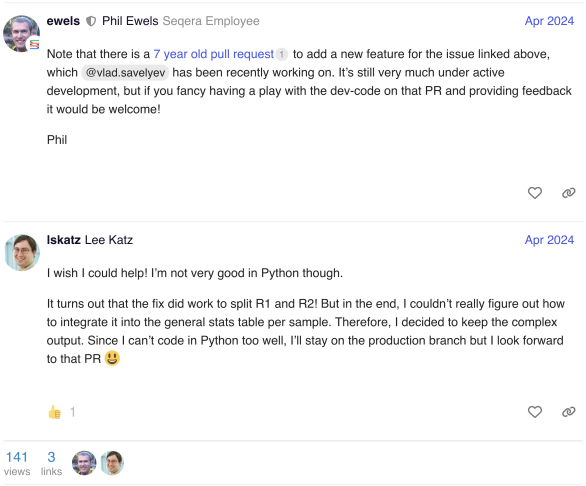

Fig. 2: Forum post in which a Nextflow contributor is asked to review a pull request but declines due to technical limitations.

This story is common in workflow-driven science. Workflow users and maintainers depend on a chain of tools, reports, and wrappers. Nextflow pipelines often integrate Python tools like MultiQC for reporting, and small changes to those tools can unblock useful features downstream. Yet review capacity is limited, release cycles are tight, and potential reviewers may lack the language expertise that the change requires. In this case, the maintainer was ready to move forward and actively asked for help. The user was motivated and engaged but did not feel able to review the code. Without a timely review, the feature remained out of reach, even though a working branch existed.

Automated code review could reduce this bottleneck. An automated reviewer can scan the pull request and produce a short, plain-language summary of what changed, point out potential risks to the General Stats table, and suggest test cases that cover per-sample aggregation. It can flag style or structural issues that a maintainer would later ask to fix and highlight places where documentation needs to be updated. Most importantly, it can generate concrete, actionable comments that relate the change to the specific reporting behavior users care about. For non-expert reviewers, this kind of guidance lowers the barrier to participation; they can confirm that the automated feedback matches their expectations and leave targeted comments without reading every line of code. For maintainers, automated triage and comment drafts shorten the path to merge by catching obvious issues early and by focusing human attention where judgment is needed most.

The same pattern appears in DSL-based workflows. Nextflow scripts are written in a domain-specific language, and their correctness depends on orchestration choices that are easy to misread: process inputs and outputs, channel operations, resource declarations, and container bindings. Manual review helps catch these issues, but it is time-consuming and unevenly distributed across contributors. Prior work argues that review improves reliability and reproducibility in scientific code and offers practical checklists to make review more systematic [4], [6]. The forum example shows the gap between that ideal and day-to-day practice: maintainers need help, contributors are willing but unsure, and features wait behind reviews that never land.

This study is motivated by closing that gap. We investigate whether large language models can provide useful, domain-aware review comments for workflow ecosystems that include Nextflow pipelines and their dependent tools. Our aim is not to replace human review, but to make it easier for maintainers to ship safe changes and for users without deep programming experience to contribute. If automated review can summarize diffs, surface risks, and suggest focused fixes for DSL workflows and their companion tools, then cases like the one in Figure 2 should end with a merged pull request and a released feature, rather than with both sides waiting.

IV. Methodology

This study follows a two-phase design: we first replicate a state-of-the-art automated code review model from software engineering and evaluate its zero-shot performance on Nextflow workflows; we then adapt the model with domain-specific data and reassess performance and practitioner value. Figure 3 summarizes the experimental pipeline, from dataset construction to baseline replication, domain adaptation, and practitioner studies. All code and data were drawn from public repositories, and we scope our analysis to Nextflow to focus on a representative DSL used widely in bioinformatics and computational biology [1]. Below we describe each steps of our methodology.

A. Dataset Constructions

We constructed a domain-specific corpus by mining the top ten public Nextflow pipelines on GitHub. Table I provides the list of pipeline we extracted the data from. Using the GitHub API, we collected merged pull requests that contained both code diffs and human-written review comments. For each eligible pull request, we extracted the file's base content, the diff hunk aligned to the review thread, and the review comment text, together with lightweight metadata such as repository, file path, and timestamps. This design targets the actual unit of review—diff hunks—mirroring modern practice and the granularity used by recent automated review models [11]. Initially we extracted **3786 review comment** along with the metadata. Then, We excluded non-English comments, pure acknowledgments or social niceties, and reply-only threads without substantive feedback, yielding a high-quality collection of real review instances from active Nextflow projects with **1743 comments**. In some of the comments, the file changed was eventually removed from the pull request resulting into empty base content. These data points do not provide enough context for the LLM to generate the code review comment and therefor is removed.

TABLE I: List of Top nf-core Pipelines Used for Dataset Collection

TOP 10 SELECTED `nf-core` NEXTFLOW PIPELINES		
nf-core/rnaseq	**nf-core/sarek**	**nf-core/nanoseq**
nf-core/mag	**nf-core/rnafusion**	**nf-core/fetchngs**
nf-core/ampliseq	**nf-core/methylseq**	**nf-core/chipseq**
nf-core/atacseq		

After removing the noisy data points, We normalized whitespace and stripped repository-specific boilerplate that would otherwise inflate lexical overlap metrics without improving semantic evaluation. The result is a cleaned corpus of **1429 of Nextflow code review data** focused on substantive feedback about processes, channels, resource declarations, and orchestration. To support both adaptation and unbiased assessment, we split the cleaned corpus into training, validation, and test partitions at the pull-request level to prevent leakage across splits. We used an 80/10/10 protocol, reserving the test partition exclusively for final evaluation in both the baseline and fine-tuned settings. The validation partition guided early stopping and prompt/rubric choices for the LLM finetuning. All random splits were seeded for repeatability and documented in the replication package.

B. Selection of the Baseline Model

We selected CodeReviewer [11] as the baseline because it is the most widely used end-to-end framework trained on review-specific tasks, it operates at the diff-hunk granularity where reviews actually occur, and it is pretrained on the largest multilingual code-review corpus (27 GB across nine languages from the top 10k repositories). Unlike general code

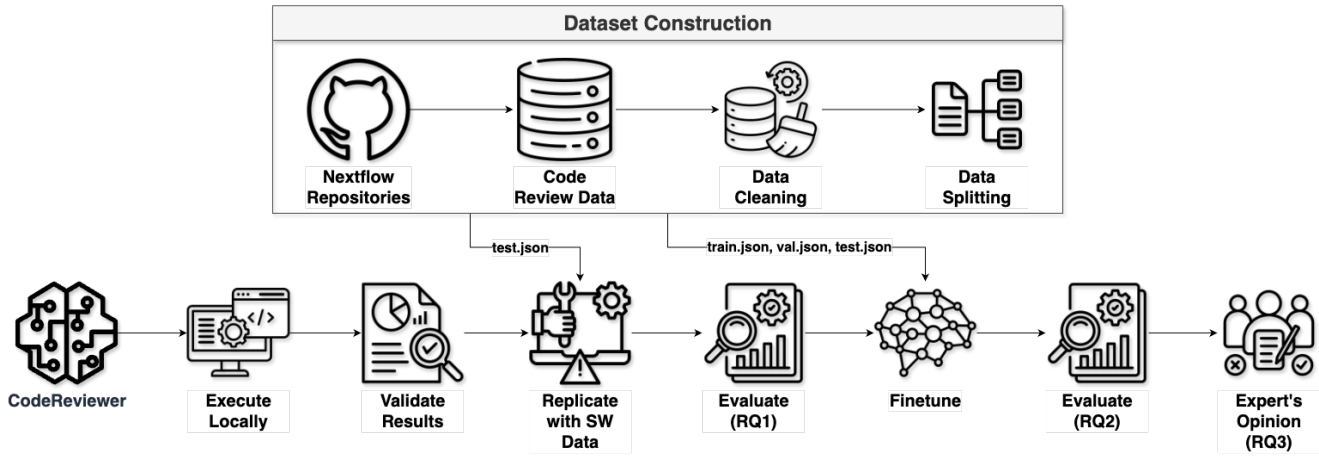

Fig. 3: Schematic diagram of the study

models, CodeReviewer's pretraining includes review-oriented objectives (diff-tag prediction, denoising code diffs and review comments, and review-comment generation), which makes it a strong candidate for transfer to new domains. At the same time, its training data and objectives are rooted in general-purpose languages, providing a fair test of domain shift when applied to Nextflow.

C. Evaluation Criteria

The baseline model only evaluated the LLM generated comment based on BLUE-4 and Human Evaluation. For a more robust evaluation, we combine complementary automated and human-centric criteria used in prior work and recommended by recent surveys [14]–[17]. These metrics collectively capture lexical accuracy, structural similarity, semantic understanding, and qualitative review quality. By using complementary evaluation methods, we ensure that both the surface form and the underlying meaning of generated comments are thoroughly assessed. Below, we provide the details of the evaluation matrices used in our study.

• **BLEU (*Bilingual Evaluation Understudy*):** BLEU measures the n-gram overlap between the generated review comment and the reference comment. It is precision-oriented, meaning it rewards outputs that match the reference in wording and structure. BLEU is effective in quantifying exact matches and is widely used in machine translation and text generation. However, it is less tolerant of paraphrasing, synonyms, or variations in phrasing, which means it primarily captures syntactic rather than semantic similarity.

• **ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*):** ROUGE is a recall-oriented metric that evaluates the overlap between generated and reference text in terms of lexical units. We use Rouge-L for our experiment. ROUGE-L measures the longest common subsequence, reflecting the extent to which the generated text preserves the overall structure of the reference.

• **BERTScore:** BERTScore measures semantic similarity by leveraging contextual embeddings from a pre-trained BERT model. It aligns each token in the generated text with the most similar token in the reference, based on cosine similarity in embedding space. This approach captures meaning beyond surface-level matches, making it well-suited for evaluating review comments that may use different wording but still convey the same intent. BERTScore is particularly valuable in domains with domain-specific terminology, where exact matches may be rare but semantic accuracy is crucial.

• **LLM-Based Evaluation (*LLaMa 3 8B-Instruct*):** This method uses a large language model as an evaluator, prompted to score generated outputs along multiple qualitative dimensions:

- **Consistency:** Logical coherence and alignment of the comment with the code context.
- **Helpfulness:** The degree to which the comment provides actionable and constructive feedback.
- **Fluency:** Grammatical correctness, readability, and natural flow of the comment.

LLM-based evaluation captures high-level, human-like quality judgments that automated similarity metrics may overlook.

• **Human Evaluation:** Conducted on 20% of the test set, human raters assess two key attributes:

- **Information:** The completeness and usefulness of the information provided in the comment.
- **Relevance:** The degree to which the comment addresses the actual issue in the code.

Human evaluation allows for nuanced judgment of domain-specific correctness, contextual appropriateness, and practical utility—factors that are especially important in DSL-based scientific workflows, where even subtle inaccuracies can have significant downstream consequences.

D. Baseline Reproduction

After finalizing the evaluation matrices, we reproduced CodeReviewer locally following the official repository and configuration from [11]. We verified the environment, tokenizer, and decoding settings, executed the authors' evaluation

pipeline on the public benchmark to sanity-check the setup, and documented any deviations needed for current library versions. Our reproduction targeted parity on the originally reported metrics (including BLEU-4 and human judgments) to ensure that any observed changes on Nextflow stem from domain differences rather than implementation drift. To validate our reproduction, we compared our scores on the original benchmark to those reported by the authors. We accepted runs whose BLEU-4 matched within a small tolerance and whose qualitative outputs were consistent with examples in the paper and repository. This step provides a reference point: if our replicated model behaves similarly on the authors' data, then performance drops on Nextflow can be interpreted as genuine domain-shift effects. Then we calculate the additional evaluation matrices (i.e. ROGUE-L, BERTScore, LLM-Evaluator) of the reproduced model.

E. Baseline Evaluation on Nextflow Data (RQ_1)

With the reproduced model fixed, we evaluated its zero-shot performance on the Nextflow test set. Inputs consisted of the file base content and the associated diff hunk; outputs were the model's generated review comments. We measured BLEU-4, ROUGE-L, and BERTScore against human references, collected LLM-judge scores for consistency, helpfulness, and fluency, and conducted the manual rating on the labeled subset. We complemented aggregate scores with error analysis focused on DSL-specific constructs—misinterpretation of channel operations, confusion between process inputs and outputs, and suggestions that violate Nextflow execution semantics to characterize failure modes unique to scientific workflows.

F. Fine-tuning on Nextflow data

We then finetuned CodeReviewer using the Nextflow training split. Fine-tuning followed the original review-comment generation objective and used early stopping on the validation partition to avoid overfitting to repository-specific phrasing. We preserved the tokenizer and decoding settings from the baseline to isolate the effect of domain adaptation. When necessary, we performed a small learning-rate sweep on the validation set and selected the setting that maximized BERTScore-F1 without harming ROUGE-L, documenting choices in the replication package. The goal of this step is to teach the model Nextflow's vocabulary, idioms, and orchestration patterns, not to change its overall architecture.

G. Evaluation and Comparison (RQ_2)

After fine-tuning, we re-evaluated the model on the same held-out test set and metrics. We compared scores before and after adaptation and examined qualitative differences in comment content. We paid special attention to specificity (e.g., naming the offending process or operator), actionability (e.g., proposing concrete channel transformations or resource directives), and alignment with workflow semantics. Improvements here indicate that domain-specific training helps the model link code diffs to appropriate review language in a DSL context.

H. Survey and Cognitive Load Experiment (RQ_3)

To assess practical value, we ran a small practitioner study (3 participants) with two complementary components. First, two authors manually rated around one-fifth of the test items on informativeness and relevance, providing a human check on the automated metrics. Second, we conducted a cognitive-load experiment where reviewers analyzed short Nextflow snippets of varying complexity under two conditions: with and without the model's review comments. We randomized item order and condition assignment to reduce learning effects. Participants rated perceived mental demand and effort on 1–5 scales after each item. In addition, we interviewed three experienced reviewers to gather qualitative feedback about usefulness, trust, failure cases, and desired integrations into existing workflows. Together, these measures estimate whether automated feedback reduces reviewer burden and helps contributors with limited programming experience participate more effectively.

I. Ethical Considerations and Availability

All data were collected from public repositories under their respective licenses. We report only aggregate results and remove personally identifying information from quoted comments where appropriate. Scripts for data collection, cleaning, splitting, evaluation, and model adaptation, along with configuration files and random seeds, are included in the replication package to support independent verification and reuse.

V. Results

This section presents our empirical findings in response to the three research questions (RQs), supported by data and evidence from our analysis.

A. Answering RQ_1

To address RQ1, we evaluated the performance of a state-of-the-art code review model, *CodeReviewer*, on a curated test set consisting of review instances extracted from real-world scientific workflows written in Nextflow—a widely used domain-specific language (DSL). For each test instance, the model generated review comments, which were then assessed using both automatic and human evaluation methods. The comparison of N-Gram based metrics (BLEU-4 and ROUGE-L) between the baseline and the replicated model is shown in Table II. The results show a decrease in BLEU-4 from 6.85 to 4.28 and similar drops in ROUGE score, indicating that the replicated model produces less surface-level overlap with the reference comments compared to the baseline.

TABLE II: RQ1 - N-Gram Matrices

Metric	Baseline	Replicated
BLEU-4	6.85	4.28
ROUGE-L	10	7

As shown in Table III, the semantic similarity metrics from BERTScore remain relatively stable between the two models, with only minor changes in precision (85 to 84), recall (84 to

81), and F1-score (84 to 82). This suggests that while exact word matches decreased, the replicated model still maintained similar semantic alignment with the references.

TABLE III: RQ1 - BERTScore

Model	Precision	Recall	F1 Score
Baseline	85	84	84
Replicated	84	81	82

The LLM-based evaluation results in Table IV provide further insight into qualitative aspects such as consistency, helpfulness, and fluency. The replicated model slightly outperformed the baseline in consistency (2.65 vs. 2.57) and helpfulness (2.78 vs. 2.64), while fluency remained nearly unchanged (4.11 vs. 4.14).

TABLE IV: RQ1 - LLM-Based Evaluation

Model	Consistency	Helpfulness	Fluency
Baseline	2.57	2.65	4.14
Replicated	2.64	2.78	4.11

Finally, the human evaluation results in Table V indicate that participants rated the replicated model lower in both information (2.37 vs. 3.60) and relevance (1.63 vs. 3.20). This implies that while the replicated model can maintain semantic similarity and reasonable fluency, it may provide less informative and relevant feedback from a human perspective compared to the baseline.

TABLE V: RQ1 - Human Evaluation

Metric	Baseline	Replicated
Information	3.60	2.37
Relevance	3.20	1.63

B. Answering RQ_2

To address RQ2, we fine-tuned the *CodeReviewer* model on a domain-specific corpus consisting of annotated review comments extracted from real-world Nextflow-based workflows. This aimed to improve the model's familiarity with the unique syntax, structure, and semantics of DSLs used in scientific workflow development.

TABLE VI: RQ2 - N-Gram Matrices

Metric	Replicated	Finetuned	% Improved
BLEU-4	4.28	23.23	442.76%
ROUGE-L	7	28	300.00%

Table VI presents the syntactic similarity results using N-Gram based metrics. The fine-tuned model shows substantial improvements over the replicated version, with BLEU-4 increasing from 4.28 to 23.23 (a 442.76% improvement) and similar percentage gains for ROUGE-L. These results indicate that fine-tuning greatly enhanced the model's ability to generate review comments that closely match the reference comments in wording and structure.

As shown in Table VII, the semantic similarity scores from BERTScore also improved, albeit with smaller percentage gains compared to syntactic measures. Precision increased from 0.84 to 0.86, recall from 0.81 to 0.86, and F1-score from 0.82 to 0.86. These results suggest that fine-tuning not only improved surface-level similarity but also enhanced the semantic alignment between generated and reference comments.

TABLE VII: RQ2 - BERTScore

Metric	Replicated	Finetuned	% Improved
Precision	84	86	2.38%
Recall	81	86	6.17%
F1 Score	82	86	4.88%

The LLM-based evaluation in Table VIII provides insight into qualitative improvements. The fine-tuned model achieved higher scores in consistency (18.11% improvement) and helpfulness (19.42%), with a marginal increase in fluency. This suggests that fine-tuning enhanced the overall usefulness and coherence of generated review comments while maintaining high language quality.

TABLE VIII: RQ2 - LLM-Based Evaluation

Metric	Replicated	Finetuned	% Improved
Consistency	2.65	3.13	18.11%
Helpfulness	2.78	3.32	19.42%
Fluency	4.11	4.13	0.49%

Finally, Table IX shows the human evaluation results. The fine-tuned model achieved higher ratings for both information (44.72% improvement) and relevance (120.86%), confirming that the generated comments were perceived as more informative and contextually appropriate. This reinforces the conclusion that fine-tuning on domain-specific DSL code reviews can significantly improve both quantitative and qualitative performance.

TABLE IX: RQ2 - Human Evaluation

Metric	Replicated	Finetuned	% Improved
Information	2.37	3.43	44.72%
Relevance	1.63	3.60	120.86%

C. Answering RQ_3

To assess the practical value of automated code review in the context of DSL-based scientific workflow, we conducted a survey with code review experts and a controlled cognitive study. First, we conducted a survey to three experienced code review practitioners who have expertise in reviewing

DSL-based scientific workflows. The survey collected their perceptions of current review practices, challenges faced when reviewing workflow code, and the perceived need for automation in this domain.

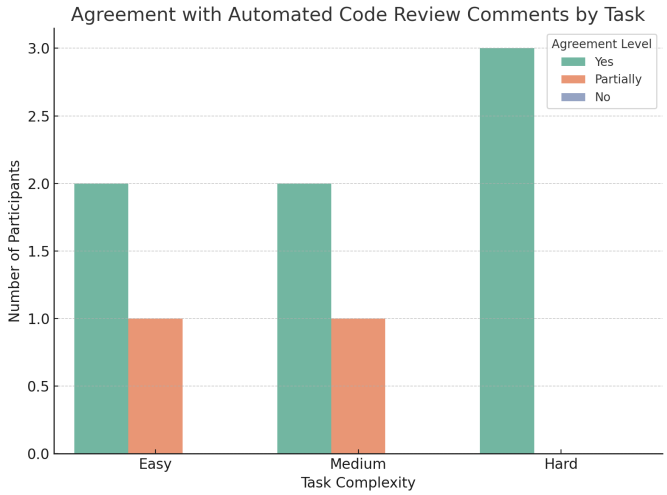

Fig. 4: Agreement Across Task Types.

Second, we ran a cognitive effort experiment in which participants reviewed workflow code snippets of varying complexity (easy, medium, and hard). Each participant completed the review tasks both *with* and *without* LLM-generated review comments. This experimental design allowed us to measure the impact of automation on mental demand, perceived effort, and agreement with the provided suggestions.

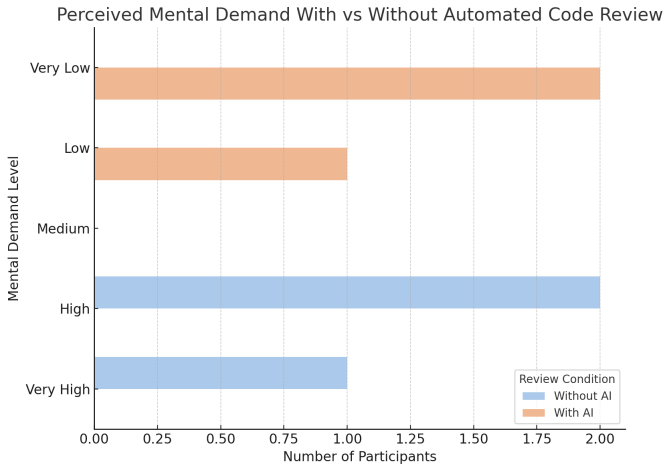

Fig. 5: Mental Demand Comparison Chart.

Agreement on LLM-Generated Comments: Participant feedback indicated strong acceptance of the LLM-generated review comments across all task complexities. As shown in Figure 4, most participants either fully agreed or partially agreed with the automated suggestions. Agreement levels were highest for the hard task condition, suggesting that automation was particularly valued when reviewing more complex workflow snippets. This trend highlights the perceived usefulness and reliability of LLM-generated comments in supporting scientific workflow code reviews.

Perceived Mental Demand and Effort: Participants consistently indicated that manual review of scientific workflow code—particularly in unfamiliar DSLs like Nextflow—was cognitively demanding. This was especially evident in the medium and hard tasks, where they had to infer intent, decode unfamiliar syntax, and reason about task dependencies. However, when supported by LLM-generated comments, participants experienced a noticeable reduction in mental workload across all task levels. The automated suggestions helped participants focus on specific issues, clarified ambiguous code segments, and reduced the need for deep inspection or domain expertise.

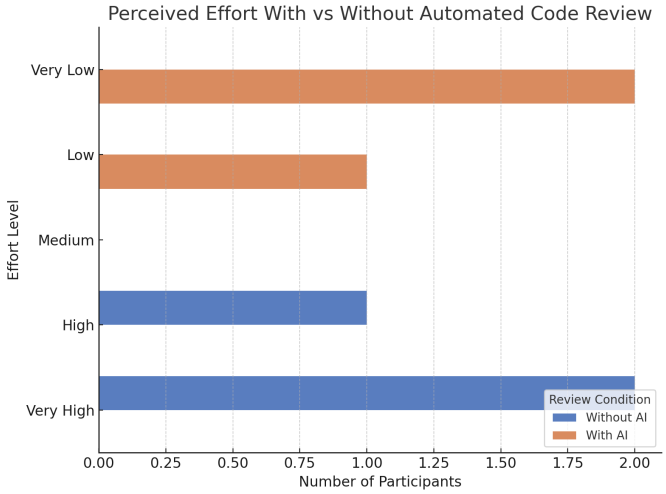

Fig. 6: Effort Comparison Chart.

Figure 5 compares perceived mental demand with and without LLM-generated comments. Across all complexity levels, mental demand scores shifted from the *High/Very High* range without automation to the *Low/Very Low* range with automation, highlighting the potential of automated assistance to reduce cognitive load. Similarly, Figure 6 compares perceived effort between the two conditions. Without automation, participants reported high effort levels across tasks, while with automation, effort ratings dropped substantially—often into the *Low* or *Very Low* categories. This suggests that automated comments not only reduce mental strain but also make the review process less labor-intensive.

VI. Discussion

A. Limitations of General-Purpose Code Review Models

Our baseline experiments with the CodeReviewer model showed that general-purpose LLMs struggle to generate effective review comments for scientific workflow code. While the model demonstrated some ability to produce syntactically valid comments, its overall performance was suboptimal. Human evaluation indicated that, although the generated comments were sometimes clear, they often lacked sufficient domain-specific depth and actionable guidance.

These limitations stem from several interconnected factors. First, general-purpose LLMs have inadequate exposure to the syntax, structure, and semantics of domain-specific languages like Nextflow, which follows a declarative, dataflow-oriented design. Without familiarity with constructs such as process blocks, channel operations, and execution directives, the model struggles to produce contextually meaningful feedback. Second, scientific workflows place unique emphasis on reproducibility, data provenance, and modular composition—goals rarely addressed in mainstream code review datasets—leading to gaps in the model's ability to assess such requirements. Finally, there is a lack of publicly available, standardized code review data for DSL-based workflows, meaning the model has little exposure to the review styles and quality standards expected in this domain. As a result, it often defaults to generic, non-actionable suggestions.

B. Benefits of Fine-Tuning on Domain-Specific Data

Our baseline experiments with the *CodeReviewer* model showed that while general-purpose LLMs can produce syntactically valid review comments, they often lack the depth, specificity, and actionable guidance needed for scientific workflow code. Part of the problem stems from the model's limited exposure to the syntax, structure, and semantics of domain-specific languages like Nextflow, which follows a declarative, dataflow-oriented design. Without familiarity with constructs such as process blocks, channel operations, and execution directives, the model struggles to provide contextually meaningful feedback. Scientific workflows also place a strong emphasis on reproducibility, data provenance, and modular composition—priorities that are rarely addressed in the datasets these models are trained on. The lack of publicly available, standardized review data for DSL workflows further limits the model's understanding of what high-quality feedback should look like.

C. Perceived Impact on Developer Experience and Workload

Beyond accuracy and performance, we explored how automated code review impacts real users—particularly in terms of perceived workload and accessibility. Participants reviewed workflow snippets of varying complexity (easy, medium, hard), first without assistance and then with LLM-generated comments. Across all tasks, users reported lower mental demand and effort when supported by the automated review system.

- For simple tasks, automation helped clarify small inconsistencies and structure, improving confidence and reducing overthinking.
- For medium and hard tasks, participants expressed appreciation for the contextual suggestions, especially when debugging logic or aligning to project conventions.
- Importantly, even users with limited programming experience found the automated suggestions helpful, reinforcing the idea that LLM-based code review can lower the entry barrier for contributors.

Moreover, participants largely agreed with the feedback produced by the fine-tuned model, indicating trust in the system's ability to assist—not replace—human reviewers. This trust is a critical factor in adoption, especially in collaborative environments like scientific computing, where code often undergoes community scrutiny and needs to adhere to rigorous quality standards. These findings suggest that LLM-assisted review tools can 1) reduce the cognitive and time-related cost of code evaluation, 2) support novice and interdisciplinary contributors in understanding best practices, 3) encourage broader community participation in scientific software development, and 4) complement human reviewers rather than displacing them

D. Data Quality Impacts Performance

Another challenge came from the quality of the training data itself. Many original review comments were cryptic, lacked actionable suggestions, or were phrased in a way that was difficult for the model to learn from. This noisiness in the data meant that the model's context window was often filled with less relevant information, making it harder to focus on the parts of the code and diff that truly mattered. This likely explains why n-gram metrics improved drastically (due to learned phrasing) while semantic/helpfulness scores improved moderately. In later experiments, we found that cleaning these comments and the base content improved performance across all metrics (by approximately 3–7%) by reducing noise and allowing the model to attend to more relevant details—a finding that highlights just how important data quality is in this setting.

VII. Implications

A. Implications Workflow Developers & Reviewers

The results of this study indicate that fine-tuned large language models (LLMs) can meaningfully support practitioners in the scientific workflow domain, particularly those working with domain-specific languages (DSLs) like Nextflow. For developers—especially researchers or domain scientists without formal software engineering backgrounds—automated code review tools provide actionable feedback on common mistakes, stylistic issues, and best practices that might otherwise go unnoticed. Developers can utilize the model to get feedback on their code, where as the reviewers can get informed suggestions from the automated review tool before providing code review comments.

B. Implications for Researchers

This work is among the first to study automated code review in the scientific workflow domain. It contributes a fine-tuned model tailored to a domain-specific language and a curated dataset of real review diffs and comments, packaged with a replicable pipeline. The results are encouraging, but they also reveal clear avenues for progress. Researchers can build directly on the released model and data to test new adaptation strategies, compare architectures, and refine evaluation methods for DSL settings.

VIII. Threats to Validity

Despite careful design and execution, certain factors may limit the interpretation and generalizability of this study's findings. Below, we outline potential threats to validity and the measures taken to mitigate them.

A. Internal Validity

The primary internal validity concern lies in the human evaluation process. All qualitative assessments of generated review comments were conducted by the authors, introducing the risk of subjective bias in scoring relevance, informativeness, and helpfulness. Although independent scoring by multiple evaluators was employed and disagreements were resolved through structured discussion, complete objectivity cannot be guaranteed. We calculated inter-rater reliability using Cohen's Kappa ($\kappa = 0.79$), which indicates substantial agreement, but some variability in judgment is inevitable in subjective evaluations. Another factor potentially affecting internal validity is the limited availability of experienced workflow developers and reviewers, which may have restricted the diversity of perspectives represented in the qualitative feedback.

B. External Validity

The dataset used for fine-tuning consisted of approximately 1,100 annotated examples sourced from only 10 publicly available repositories, all written in the Nextflow DSL. While this design ensured strong domain alignment, it inherently restricts the applicability of our findings to other scientific workflow management systems such as Snakemake, Galaxy, or CWL-based platforms. Moreover, many of the selected workflows came from well-maintained repositories that follow community standards, potentially biasing the model toward more formal review styles. The survey and cognitive workload study also involved a relatively small participant pool due to time constraints and delays in obtaining ethics approval. As a result, the sample may not fully represent the diversity of the global scientific workflow developer community.

C. Construct Validity

Construct validity concerns arise from the limited representation of experienced practitioners in our participant pool. Both the survey and the expert evaluation tasks included individuals with varying levels of familiarity with Nextflow, which may have influenced their perceptions of mental demand, effort, and agreement with automated comments. Additionally, the subjective nature of "helpfulness" and "informativeness" scores, even when well-defined, leaves room for variation in interpretation between evaluators. These factors could impact the extent to which our measured constructs truly reflect the underlying phenomena we intended to capture.

D. Conclusion Validity

Our experimental setup replicated the *CodeReviewer* architecture, meaning any inherent limitations in the base model—such as prompt design, context window size, model capacity, and the composition of pre-training data—may have influenced performance outcomes. For example, biases present in the original pre-training corpus could persist in our fine-tuned model. Furthermore, given that fine-tuning was performed on a relatively small dataset, improvements may reflect overfitting to domain-specific patterns rather than a broad increase in general review capabilities. As such, while our observed performance gains are substantial, they should be interpreted with caution until validated on a wider variety of DSLs and workflow contexts.

IX. Conclusion and Future Work

A. Conclusion

This study investigated the potential of large language models (LLMs) to support code review processes in scientific workflows written in domain-specific languages (DSLs), with a primary focus on Nextflow. The results show that while general-purpose code review models can generate syntactically valid comments, they often lack the domain-specific depth and actionable feedback needed for high-quality scientific workflow reviews. Fine-tuning the model on a curated, domain-specific dataset significantly improved both quantitative and qualitative performance, underscoring the importance of adapting LLMs to specialized technical domains.

Additionally, findings from the cognitive workload study indicate that LLM-generated comments can reduce perceived mental demand and effort, particularly in more complex review scenarios. These results highlight the promise of AI-assisted tools in lowering the cognitive barriers for practitioners and enhancing the accessibility of code review in scientific programming.

B. Future Work

While the findings are promising, several limitations remain, primarily the restricted dataset size and scope, the reliance on human evaluations subject to bias, and the focus on a single DSL. To address these limitations and extend this research, we propose the following directions:

- **Curate a larger and more diverse dataset** by incorporating annotated review data from multiple DSLs and workflow systems, such as Snakemake, Galaxy, and CWL, to enhance model generalizability.
- **Integrate domain knowledge during pre-training** as well as fine-tuning, enabling the model to develop a deeper structural and semantic understanding of workflow languages.
- **Explore additional automated review capabilities**, such as predicting the necessity of a review for a given code segment and automatically generating code modifications or fixes in response to detected issues.

Pursuing these directions will contribute to building AI-assisted review systems that are more accurate, context-aware, and adaptable across the diverse landscape of scientific workflow development.

REFERENCES

[1] P. Di Tommaso, M. Chatzou, E. Floden, P. P. Barja, E. Palumbo, and C. Notredame, "Nextflow enables reproducible computational workflows," *Nature Biotechnology*, vol. 35, no. 4, pp. 316–319, 2017.

[2] J. Köster and S. Rahmann, "Snakemake—a scalable bioinformatics workflow engine," *Bioinformatics*, vol. 28, no. 19, pp. 2520–2522, 2012.

[3] Galaxy Community, "The galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2022 update," *Nucleic Acids Research*, vol. 50, no. W1, pp. W345–W351, 2022.

[4] E. R. Ivimey-Cook, J. L. Pick, K. R. Bairos-Novak, A. Culina, E. Gould, M. Grainger, B. M. Marshall, D. Moreau, M. Paquet, R. Royauté, A. Sánchez-Tójar, I. Silva, and S. M. Windecker, "Implementing code review in the scientific workflow: Insights from ecology and evolutionary biology," *Journal of Evolutionary Biology*, vol. 36, no. 10, pp. 1347–1356, 2023.

[5] A. Rokem *et al.*, "Ten simple rules for scientific code review," *PLOS Computational Biology*, vol. 20, no. 9, p. e1012375, 2024.

[6] F. Hillemann, J. B. Burant, A. Čulina, and S. J. G. Vriend, "Code review in practice: A checklist for computational reproducibility and collaborative research in ecology and evolution," 2025, preprint. [Online]. Available: https://doi.org/10.32942/X21S7H

[7] N. Eisty and J. C. Carver, "Developers' perception of peer code review in research software development," *Empirical Software Engineering*, vol. 27, no. 5, p. 111, 2022.

[8] M. Petre and G. Wilson, "Code review for and by scientists," *arXiv*, 2014.

[9] Y. Patel, T. Tran, S. Enoch, S. Rafique, J. Eizenga, K. Ellrott *et al.*, "Nftest: Automated testing of nextflow pipelines," *Bioinformatics*, vol. 40, no. 2, p. btae081, 2024.

[10] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proceedings of the 2013 International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721.

[11] Z. Li, S. Lu, D. Guo, N. Duan, S. Jannu, G. Jenks, D. Majumder, J. Green, A. Svyatkovskiy, S. Fu, and N. Sundaresan, "Automating code review activities by large-scale pre-training," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022)*. ACM, 2022, pp. 1–13. [Online]. Available: https://doi.org/10.1145/3540250.3549081

[12] Y. Hong, C. Tantithamthavorn, P. Thongtanunam, and A. Aleti, "Commentfinder: A simpler, faster, more accurate code review comments recommendation," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 507–519.

[13] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, "Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning," *arXiv*, 2023.

[14] R. Tufano, O. Dabić, A. Mastropaolo, M. Ciniselli, and G. Bavota, "Code review automation: Strengths and weaknesses of the state of the art," *IEEE Transactions on Software Engineering*, vol. 50, no. 2, pp. 338–353, 2024.

[15] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.

[16] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[17] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *International Conference on Learning Representations (ICLR)*, 2020.

[18] S. Musslick, L. K. Bartlett, S. H. Chandramouli, M. Dubova, F. Gobet, T. L. Griffiths, J. Hullman, R. D. King, J. N. Kutz, C. G. Lucas, S. Mahesh, F. Pestilli, S. J. Sloman, and W. R. Holmes, "Automating the practice of science: Opportunities, challenges, and implications," *Proceedings of the National Academy of Sciences*, vol. 122, no. 5, p. e2401238121, 2025. [Online]. Available: https://www.pnas.org/doi/10.1073/pnas.2401238121

[19] Z. Liu, Y. Chai, and J. Li, "Toward automated simulation research workflow through llm prompt engineering design," *Journal of Chemical Information and Modeling*, vol. 65, no. 1, pp. 114–124, 2025. [Online]. Available: https://doi.org/10.1021/acs.jcim.4c01653

[20] O. Yildiz and T. Peterka, "Do large language models speak scientific workflows?" *arXiv preprint arXiv:2412.10606*, 2025. [Online]. Available: https://arxiv.org/abs/2412.10606

[21] J. Y. Shih, V. Mohanty, Y. Katsis, and H. Subramonyam, "Leveraging large language models to enhance domain expert inclusion in data science workflows," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*. New York, NY, USA: ACM, 2024. [Online]. Available: https://doi.org/10.1145/3613905.3651115

[22] T. Procko, A. Davidoff, T. Elvira, and O. Ochoa, "Towards improved scientific knowledge proliferation: Leveraging large language models on the traditional scientific writing workflow," *SSRN Electronic Journal*, 2023, preprint. [Online]. Available: https://ssrn.com/abstract=4594836

[23] J. Xu, W. Du, X. Liu, and X. Li, "Llm4workflow: An llm-based automated workflow model generation tool," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, 2024, pp. 2394–2398. [Online]. Available: https://doi.org/10.1145/3691620.3695360

[24] H. Y. Lin, P. Thongtanunam, C. Treude, and W. Charoenwet, "Improving automated code reviews: Learning from experience," in *Proceedings of the 21st International Conference on Mining Software Repositories (MSR '24)*. New York, NY, USA: ACM, 2024, pp. 278–283. [Online]. Available: https://doi.org/10.1145/3643991.3644910

[25] P. Ewels and L. Katz, "Fastqc read quality scores in general stats table," Seqera Community Forum, Apr 2024, discussion thread; accessed 2025-08-08. [Online]. Available: https://community.seqera.io/t/fastqc-read-quality-scores-in-general-stats-table/673

[26] MultiQC Contributors, "Multiqc: Pull request #576," GitHub, 2017, open PR with ongoing updates; linked from Seqera forum (Apr 2024); accessed 2025-08-08. [Online]. Available: https://github.com/MultiQC/MultiQC/pull/576